TECHNICAL REPORT

How the Platform Works Architecture and Design of a Secure Multi-Tenant Retrieval-Augmented AI Assistant Platform

Mohammed Amine Ben Ghorbal

2026

Contents

List of Abbreviations **v**

Introduction **1**

1 What the Platform Does **3**
1.1 The Problem 3
1.2 Who Uses the Platform 3
1.3 The Four Pillars of the Solution 4

2 Architecture Overview **7**
2.1 Physical Architecture 7
2.2 Logical Architecture 8
2.3 Data Model 9
2.4 Technology Stack 9

3 Identity, Tenancy and Isolation **11**
3.1 Authentication and Tenant Resolution 11
3.2 Physical Tenant Isolation 12
3.3 A Dedicated Qdrant Instance per Company 12

4 The AI Provider Layer **14**
4.1 A Runtime-Editable Provider Registry 14
4.2 Encryption of API Keys 14
4.3 Routing, Rate Limiting and the Circuit Breaker 14

5 The Grounded Answer Pipeline **16**
5.1 From a Question to a Grounded Answer 16
5.2 Ephemeral Upload versus Audited Ingestion 17
5.3 Asking in a Language the Knowledge Base Doesn't Use 17
5.4 The Two Angular Front Ends 18

6 Safety and Governance **20**
6.1 The Safety Gate: Filter, then Classifier 20

6.2 Token-Cap Delegation 21
6.3 Self-Service Token Requests 21

7 Observability, Feedback and Extensibility 23
7.1 Answer-Quality Feedback Dashboard 23
7.2 Tracing and Prompt Versioning with MLflow 24
7.3 Agentic Admin Tooling via the Model Context Protocol 24

Conclusion 26

References 27

List of Figures

1 One question, and every subsystem it passes through end to end. 1

1.1 RAG in plain terms: the answer is grounded in retrieved documents, not memory. 5
1.2 Multi-tenancy in plain terms: each company's documents stay walled off from the rest. 5

2.1 Physical architecture: the Angular front end talks only to the Flask API. . 8
2.2 Logical architecture: routes call services, which resolve identity before touching data. 9

3.1 From login to retrieval: the company id is resolved server-side, never from the client. 12

4.1 `llm_router.py` is the only module in the codebase allowed to call a model. 15

5.1 The full `/api/query` pipeline, step by step. 16
5.2 Ephemeral upload versus audited, permanent ingestion. 17
5.3 Only the search query is translated; the model answers in the user's own language. 18

6.1 Every piece of text passes through the Filter, then the Classifier, before reaching a model. 20
6.2 Caps flow down the ladder; raise requests flow back up it. 21

7.1 Two always-on paths, tracing and feedback, both keyed to the answering provider. 23
7.2 The MCP server is a thin wrapper: an agent can do only what an admin already could. 25

List of Tables

1.1 Traditional setup versus this platform 6

2.1 Technology stack 10

List of Abbreviations

API	Application Programming Interface
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
KB	Knowledge Base
LLM	Large Language Model
MCP	Model Context Protocol
NER	Named Entity Recognition
PII	Personally Identifiable Information
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
RPD	Requests Per Day
RPM	Requests Per Minute
SDK	Software Development Kit
SPA	Single Page Application
SQL	Structured Query Language
SSE	Server-Sent Events
TTL	Time To Live
URL	Uniform Resource Locator

Introduction

A generic AI chatbot, trained only on public data, cannot answer questions about a specific company's products, contracts, or internal procedures, and may *invent* a plausible but incorrect answer instead. **Retrieval- Augmented Generation (RAG)** closes that gap: before the model answers, the system *retrieves* the most relevant passages from the company's own documents and hands them to the model as context, so the answer is *grounded* in verifiable material rather than guessed.

This platform puts RAG at the service of a **multi-tenant** system: one running application serving several customer companies at once, each with a strictly private knowledge base no other company can see. An administrator decides, at runtime and without redeploying anything, which AI models are available and to whom, and a safety gate protects both the users and the shared infrastructure several tenants draw on at once.

This report is not a project diary; it explains **how the finished system works**: what each moving part does, how the parts connect, and, where a design choice was not the obvious one, *why*. The chapters are organized around the platform's own concerns rather than a timeline. Figure 1 is the map to keep in mind while reading: one question, and every subsystem it passes through before an answer comes back.

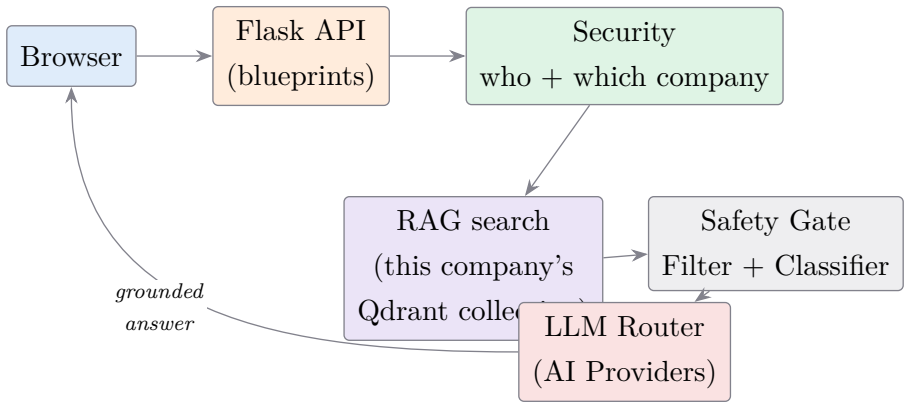


Figure 1: One question, and every subsystem it passes through end to end.

- **Chapter 1**: the problem and the shape of the solution.
- **Chapter 2**: the physical and logical architecture.
- **Chapter 3**: identity and tenant isolation.
- **Chapter 4**: the AI provider layer, key encryption, routing and rate limiting.
- **Chapter 5**: the grounded-answer pipeline end to end.

- **Chapter 6**: safety, governance, and token-budget delegation.
- **Chapter 7**: tracing, feedback, and agent-facing tooling.

A short conclusion pulls the pieces back into one picture and points at what is deliberately left open for later.

Chapter 1

What the Platform Does

In this chapter

1. The problem: generic chatbots don't know a company's own documents
2. Who uses the platform, and for what
3. The four pillars of the solution

1.1 The Problem

Support for business customers is traditionally handled by human agents, at best assisted by a generic chatbot on fixed scripts or a single general-purpose model. That setup has four recurring weaknesses:

- answers are **not grounded** in the customer company's own documentation: the assistant either does not know the answer or, worse, fabricates one (a phenomenon known as *hallucination*);
- each customer company that wants its own assistant traditionally needs a **separate deployment**, which multiplies operating cost;
- the choice of AI model is usually **hard-coded**: changing or adding a model means editing configuration files and redeploying the application;
- there is **no systematic protection** of sensitive data (personal information appearing in documents) nor of the shared API keys that free-tier AI providers rate-limit aggressively.

1.2 Who Uses the Platform

The platform recognizes three kinds of actors:

- **Customer**: belongs to exactly one *company* (tenant), chats only against that company's documents, may attach a file, and picks which AI model answers.

- **Company administrator**: manages their own company's users and token budget, and approves or forwards token-cap requests.
- **Super administrator**: platform staff, not tied to any company. Manages the AI model catalogue, ingests documents, edits safety rules, and reviews usage and feedback across every tenant.

1.3 The Four Pillars of the Solution

The platform is built around four pillars, each of which gets its own chapter later in this report:

- **Grounded answers**: every response is built from passages retrieved from the asking company's own knowledge base (Figure 1.1), with sources shown and a relevance threshold that makes the assistant say "I don't have that information" rather than guess.
- **Strict tenant isolation**: each company's documents live in a *physically separate* search index (Figure 1.2); there is no code path that reads across companies.
- **Runtime-editable AI registry**: an administrator adds, enables or disables AI models without any redeployment, and a customer picks which one answers their question.
- **Governance and safety**: a two-stage content gate redacts personal data and blocks prompt-injection, while shared-key rate limiting and per-company usage caps protect the infrastructure that every tenant relies on.

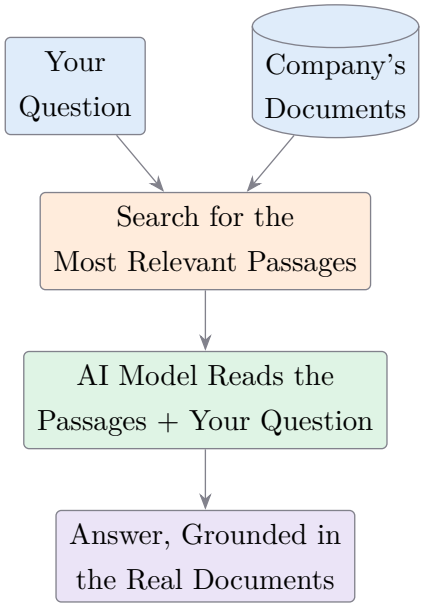


Figure 1.1: RAG in plain terms: the answer is grounded in retrieved documents, not memory.

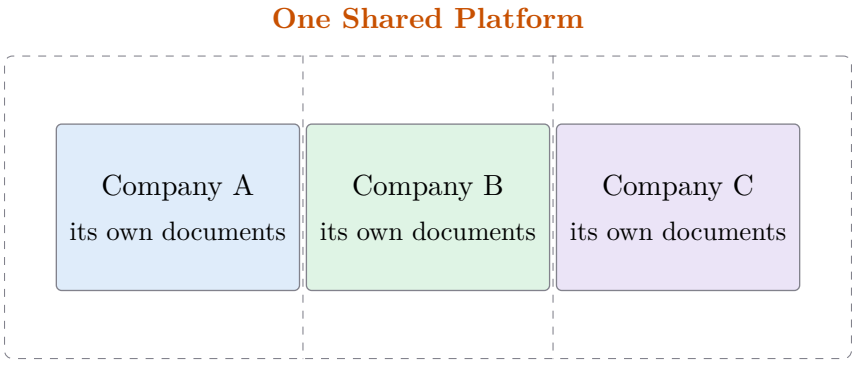


Figure 1.2: Multi-tenancy in plain terms: each company's documents stay walled off from the rest.

Table 1.1 summarizes the shift from the traditional setup to this platform.

Criterion	Traditional setup	This platform
Answer accuracy	Generic / possible hallucination	Grounded in company documents
Multi-customer	One deployment per customer	One platform, many tenants
Model choice	Hard-coded, needs redeploy	Runtime-editable registry
Data protection	None systematic	PII redaction + injection blocking
Cost control	None	Shared-key limits + per-company caps

Table 1.1: Traditional setup versus this platform

Chapter 2

Architecture Overview

In this chapter

1. Physical architecture: the three tiers
2. Logical architecture: the four layers
3. Data model: how the core tables relate
4. Technology stack

2.1 Physical Architecture

The platform follows a classic three-tier, client-server organization (Figure 2.1):

- a **presentation tier**: an Angular Single Page Application with separate shells for login, customer chat, and admin;
- an **application tier**: a Flask REST API secured by JSON Web Tokens, so the server never has to keep session state in memory;
- a **data tier**: a relational database (SQLite in development, PostgreSQL in production) for structured data, and **Qdrant**, a vector database, for document embeddings (each passage stored as numbers capturing its meaning, so similar-meaning passages surface even without shared words);
- **external AI providers**: reached through a single LLM router module that holds the encrypted keys (Chapter 4).

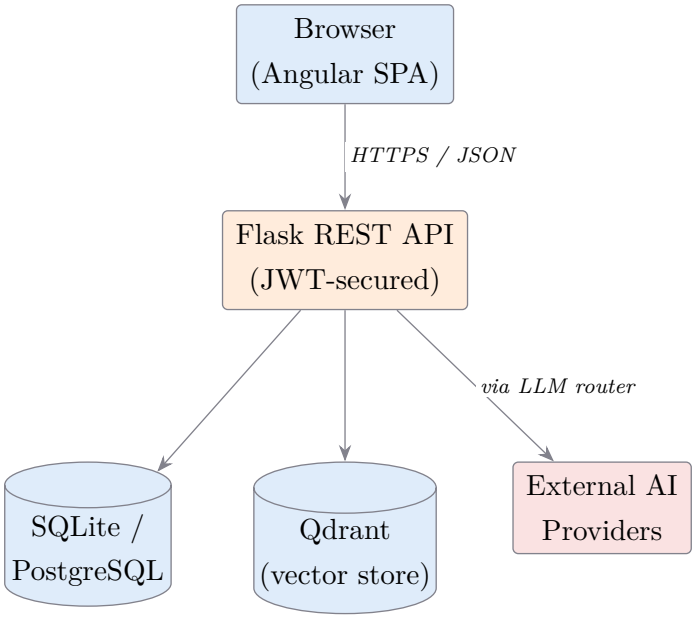


Figure 2.1: Physical architecture: the Angular front end talks only to the Flask API.

2.2 Logical Architecture

Internally, the backend is organized into layers, each with a single responsibility, so that every sensitive operation passes through exactly one auditable chokepoint (Figure 2.2):

- **Route layer (blueprints):** thin HTTP handlers grouped by topic: `auth`, `query`, `providers`, `documents`, `admin`, `users`, `token_requests`, `conversations`, `account`. A route decodes the request, calls a service, and formats the response. It does not itself decide anything sensitive.
- **Service layer:** the reusable logic. `rag.py` is the only module that talks to Qdrant; `llm_router.py` is the only module that calls an LLM; `usage.py` enforces rate limits and accounting; `screening.py` composes the Filter/Classifier safety gate; `crypto.py` encrypts provider keys; and `chunking.py`, `translation.py` and `tracing.py` handle document splitting, multilingual queries, and MLflow observability.
- **Security layer:** `security.py` decides who the caller is and which company they belong to, by re-reading the user's row from the database on every request rather than trusting the token's claims for authorization.
- **Data-model layer:** `models.py` declares the database tables: `Company`, `User`, `LlmProvider`, `Message`, `TokenUsage`, `TokenRequest`, `PiiRule`, and the document/audit tables.

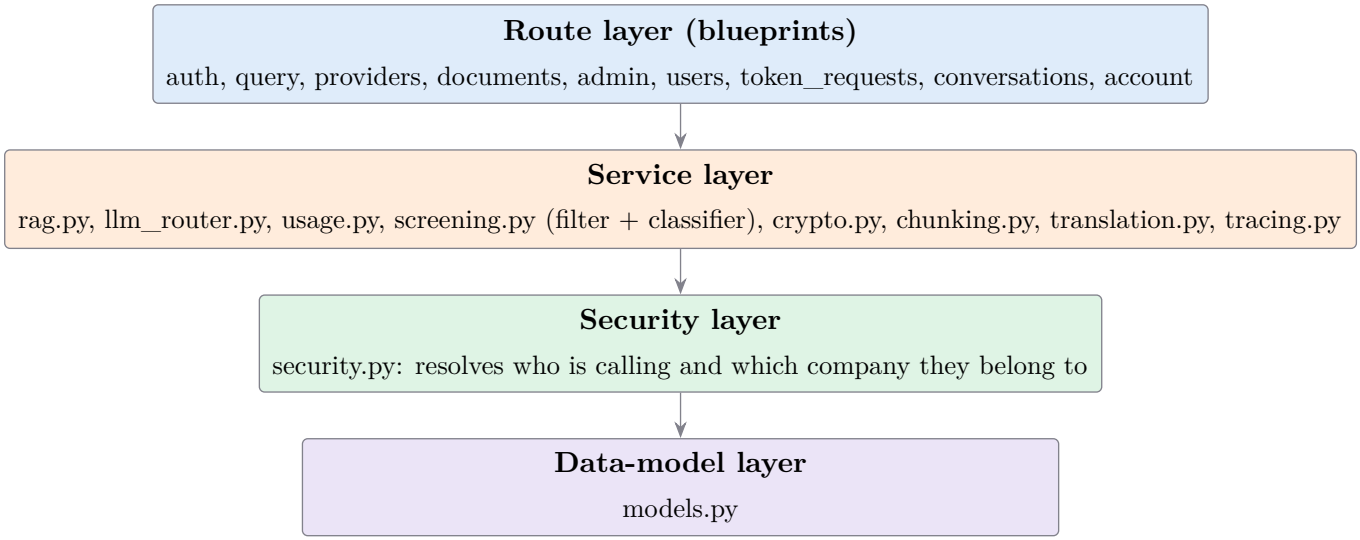


Figure 2.2: Logical architecture: routes call services, which resolve identity before touching data.

This layout embodies the design's golden rule: *routes are thin, services are smart, Qdrant access is confined to* `rag.py`*, and every outbound LLM call goes through* `llm_router.py`. A bug or a policy change in any of those concerns has exactly one place to be fixed and audited.

2.3 Data Model

Underneath the four layers sit the tables that give the whole platform its shape. **??** shows the relationships that recur throughout this report, rather than every column of every table:

- `Company` is the tenant at the center of everything: it owns its `User`s outright, and can *optionally* scope a `LlmProvider` or `PiiRule` row to itself, where `NULL` means "visible to every company," not "belongs to no one."
- `Message` is written by a `User` and separately records *which* `LlmProvider` produced it, the join key the feedback dashboard (Chapter 7) depends on.
- `TokenRequest` and `TokenUsage` both hang off `User`: one is the ask side of Chapter 6's governance system, the other is the accounting Chapter 4's provider layer checks before every call.

2.4 Technology Stack

Table 2.1 lists the main technologies behind the platform and the role each one plays.

Tool / Technology	Role in the platform
Python 3	Backend programming language
Flask	REST API framework (application tier)
Qdrant	Vector database for document embeddings
fastembed	Local embedding model (turns text into vectors)
LiteLLM	Unified SDK to call many LLM providers through one interface
JWT (`flask-jwt-extended`)	Stateless authentication
Fernet	Symmetric encryption of provider API keys at rest
Microsoft Presidio	NER-based personal-data detection in the safety gate
MLflow	Answer tracing and system-prompt versioning
SQLite / PostgreSQL	Relational database (development / production)
Angular	Frontend Single Page Application (standalone components, signals)
TypeScript	Frontend programming language
Model Context Protocol	Standard tool-calling protocol used by the admin MCP server

Table 2.1: Technology stack

Chapter 3

Identity, Tenancy and Isolation

In this chapter

1. Authentication and server-side tenant resolution
2. Physical isolation: one Qdrant collection per company
3. Going further: a dedicated Qdrant instance per company

3.1 Authentication and Tenant Resolution

Login verifies the password and issues a JWT whose claims include the user's role and company id. Those claims, however, are used only for frontend routing: they are never trusted for authorization. On every subsequent request, `security.py` decodes the token, then **re-reads the user's row from the database** by the id in the token before deciding anything: if a user's role or company were changed in the meantime, the change takes effect immediately, rather than only once their existing token expires.

`current_company_id()` is the one real chokepoint: for a company-bound user it reads `user.company_id` straight off their own row. A super administrator has no company of their own, so that role alone may send an `X-Company-Id` header, deliberately the *only* place in the application where a company id is ever taken from the client, and only after the role check has already run. No route, anywhere, reads a company id out of a request body. Figure 3.1 summarizes the whole path, from login to a company-scoped search.

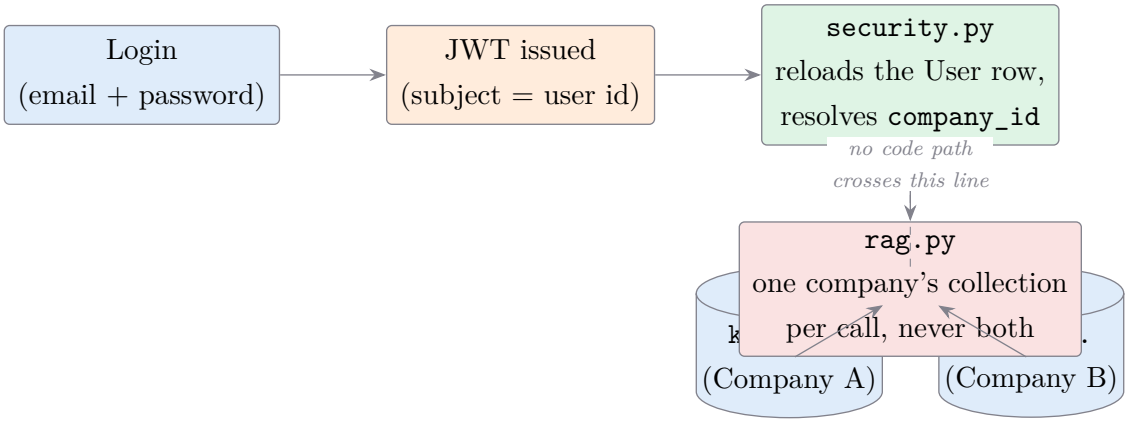


Figure 3.1: From login to retrieval: the company id is resolved server-side, never from the client.

3.2 Physical Tenant Isolation

Each company gets its own Qdrant *collection*, named from its id and a slug (for example `kb_1_telecom`). The platform deliberately does *not* store every company in one large collection filtered by a `company_id` field, since a single filter mistake could leak another company's data. With separate collections there is no code path that searches across companies: isolation is **structural**, not a condition someone could get wrong later. Access to Qdrant is confined to a single module, `services/rag.py`, so a violation would have to be a bug in that one file, and only there.

The retrieval endpoint never receives a company id from the caller: it always uses `g.company_id`, resolved server-side as above, so there is no request parameter a client could tamper with to reach another tenant's documents.

3.3 A Dedicated Qdrant Instance per Company

Collection-level isolation already means no company's questions can reach another's data. Some tenants, however, want more than logical separation: a guarantee that their vectors sit on their *own* server, for blast-radius, noisy-neighbour, or contractual reasons ("our vectors are on our own instance"). `Company.qdrant_url` supports exactly that:

$shared \Rightarrow$ `Company.qdrant_url IS NULL` $\Rightarrow$ $the platform's default$`QDRANT_URL` $\quad dedicated \Rightarrow$ `C`

Dedicated instances are *pre-provisioned* (started ahead of time as separate containers) and merely *claimed* at company-creation time rather than started on demand: letting the backend launch new containers itself would mean mounting the host's Docker socket into it, handing anything that reaches the backend the ability to start a privileged container on

the host. A fixed pool trades unlimited tenants for an honest error when it is exhausted. The claim is read from the database on every allocation rather than cached in memory, so it survives a restart and stays identical across worker processes.

It is worth being precise about what this upgrade does *not* do: it adds no isolation that was missing, since nothing could read across collections before either. It is a physical-separation and cost-isolation feature layered on a tenancy model that was already safe.

Chapter 4

The AI Provider Layer

In this chapter

1. A runtime-editable provider registry
2. Encryption of API keys
3. Routing, rate limiting and the circuit breaker

4.1 A Runtime-Editable Provider Registry

The list of available AI models is not a configuration file: it is a set of rows in a database table, `llm_providers`, editable at runtime through `GET/POST/PUT/DELETE /api/admin/llm-providers`. Adding a provider makes it appear in customers' model dropdown on their next refresh, with no redeploy, config edit, or restart. Each row carries the provider's key, base URL, model name, rate limits, a fallback priority, and generation ceilings. A single outbound module, `services/llm_router.py`, is the only code allowed to load a provider row, decrypt its key, and dispatch a call.

4.2 Encryption of API Keys

When an administrator enters a provider's secret API key, the backend encrypts it with **Fernet** symmetric encryption before saving it; it is decrypted only for the instant of the call. The public "list providers" endpoint returns names and models but *never* keys.

4.3 Routing, Rate Limiting and the Circuit Breaker

Figure 4.1 shows the path a call takes from the registry to the model that actually answers it.

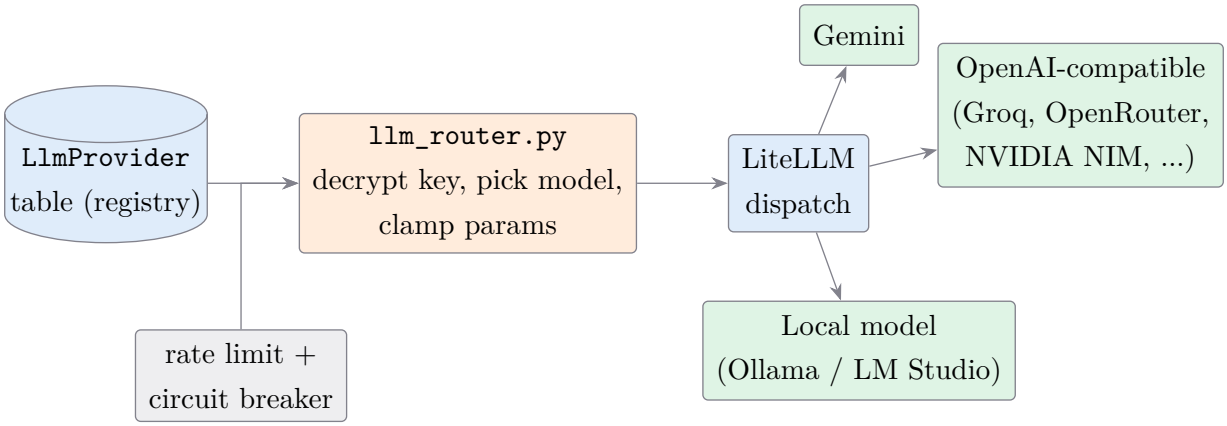


Figure 4.1: `llm_router.py` is the only module in the codebase allowed to call a model.

A customer's request carries a `provider_id` (their model choice), resolved against the enabled, company-visible rows only. Before dispatching, `llm_router.py` runs two independent safeguards, both living in `services/usage.py`:

- **Provider rate limiting**: some providers share a single key across every company; before each call, the router counts how many times *that provider* was used **across all companies** in the last minute and day, and refuses with HTTP 429 if one more call would breach the ceiling. The count is global because the shared key is what is being protected.
- **Per-company and per-user usage**: every call is logged, and an optional daily cap may be set per company and per user, so one tenant hitting its own cap never affects the others.

The distinction to hold onto: provider limiting counts a provider across all companies; usage accounting counts a company (or user) across all providers. Same events, two groupings, two jobs.

A third safeguard is a **circuit breaker**: a failed provider is marked unavailable until a cooldown passes, and greys out in the dropdown instead of being retried into more failures. On `/api/query`, the router can also **fall back** to the next enabled provider by priority when the chosen one is at capacity, reporting who actually answered. Every call also *clamps* the final temperature and max output length, with precedence *request* $\rightarrow$ *provider ceiling* $\rightarrow$ *global config*: a single enforcement point protecting a shared free-tier key from a runaway generation.

Chapter 5

The Grounded Answer Pipeline

In this chapter

1. From a question to a grounded answer, step by step
2. Ephemeral upload versus audited knowledge-base ingestion
3. Asking in a language the knowledge base doesn't use
4. The two Angular front ends

5.1 From a Question to a Grounded Answer

`POST /api/query` runs the complete pipeline shown in Figure 5.1: the question is translated for retrieval if needed, the caller's own company collection is searched, every returned chunk passes through the safety gate detailed in Chapter 6, any chunks staged from a file the user attached are merged in, a prompt is built from the system prompt plus that context, and `llm_router.complete()` (Chapter 4) produces the answer.

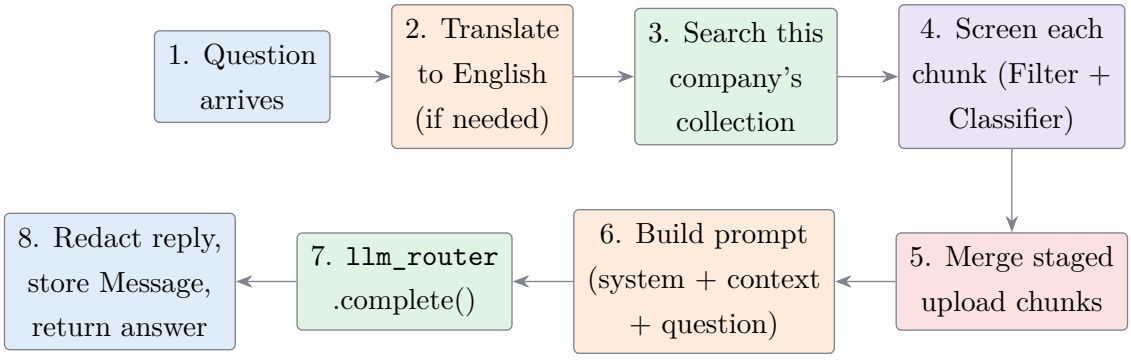


Figure 5.1: The full `/api/query` pipeline, step by step.

A relevance threshold drops weak matches, so the response can report `grounded=false` and the model answers "I don't have that information" instead of hallucinating from a barely-matching chunk. The `provider_id` is always the customer's own model choice, taken from the request body; the `company_id` never is, coming instead from the server-side resolution described in Chapter 3.

5.2 Ephemeral Upload versus Audited Ingestion

Documents enter the platform through two deliberately different routes that share the same chunking and screening code but end at very different places.

`services/chunking.py` splits text into overlapping word windows (120 words per chunk, 20-word overlap) before either route embeds it with `fastembed`, using a single locked model, `BAAI/bge-small-en-v1.5` (384-dimension, cosine similarity).

- **Ephemeral upload** (`POST /api/documents/upload`): any customer can attach a `.txt`, `.md`, `.pdf` or `.docx` file for the current conversation only. It is chunked, screened, and staged with a time-to-live, **never** written into the company's permanent index.
- **Audited ingestion** (`POST /api/documents/ingest`, administrator-only): a document is screened, chunked, embedded and permanently upserted into a chosen company's collection, recorded in an audit trail (`KbIngestion`) so every addition is traceable to the admin who made it and when.

Figure 5.2 lines the two routes up side by side: same chunking, same screening, and then a fork in how permanent the result is.

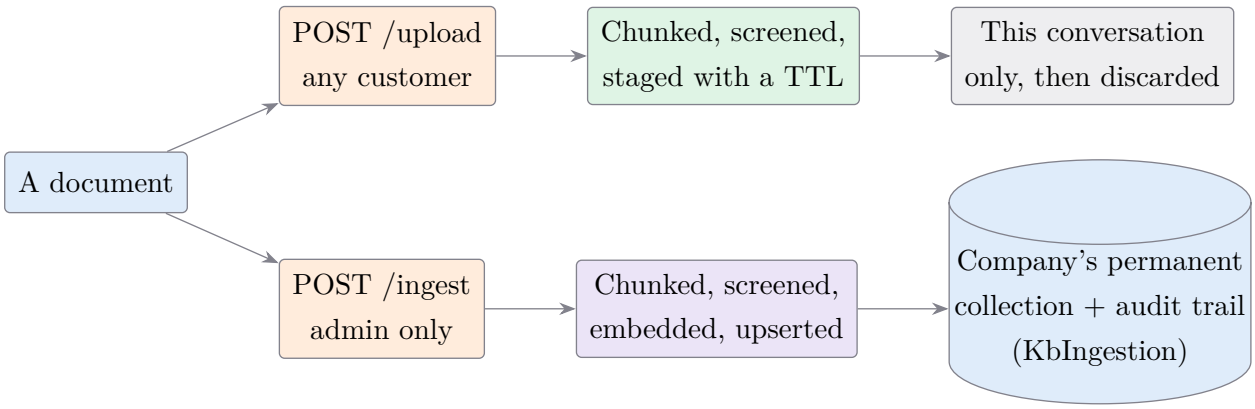


Figure 5.2: Ephemeral upload versus audited, permanent ingestion.

5.3 Asking in a Language the Knowledge Base Doesn't Use

The knowledge base is, and stays, English: every document is embedded with the same English-only model. A question asked in French, embedded by that same model, lands far from the English chunks in the vector space, so retrieval quietly finds nothing; and because the relevance threshold then drops every weak match, the failure is *silent*: the

assistant would answer “I don’t have that information” for a question the knowledge base could actually answer. Figure 5.3 shows the fix.

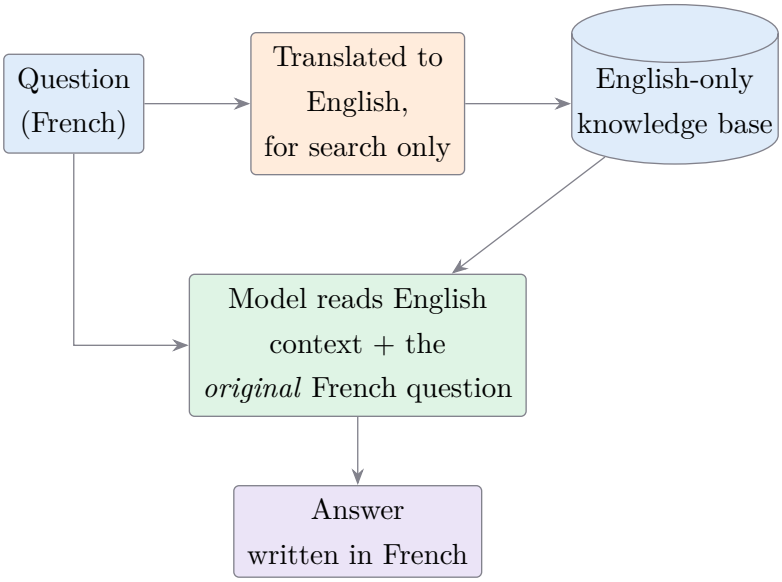


Figure 5.3: Only the search query is translated; the model answers in the user’s own language.

The fix translates the *question*, never the stored vectors, and only for the retrieval step. Translation happens strictly *before* embedding, since embedding is one-way and there is no later point where it could still happen. The answer-language instruction is a single sentence appended to the *same* completion call rather than a second “translate back” call: cheaper, and more faithful, since the model reads the English context and writes the reply in the target language in one pass. Any failure in translation returns the original, untranslated question rather than raising, since a degraded retrieval query beats failing the request outright. A companion guard, `Company.embedding_model`, records which model actually built a tenant’s collection and refuses to operate when it disagrees with the platform’s active model, even at a matching vector size: size alone cannot prove two collections were built by the *same* model.

5.4 The Two Angular Front Ends

The frontend is an Angular Single Page Application (standalone components, signals, zoneless change detection).

The **customer chat** view offers a message list, a model-selection dropdown, temperature and max-length controls, and a file-upload control for ephemeral attachments. Each answer shows which model replied, token usage, and expandable *sources* with a “redacted”

badge where PII was removed. The client upholds the tenant rule from its own side too: it never sends a `company_id`.

The **administration console** reuses the same core layer behind a role guard: the provider registry (with a connectivity "Test" button), the usage dashboard (per-company and per-provider breakdowns), and a knowledge-base tab (company picker, ingestion, and ingestion history).

Chapter 6

Safety and Governance

In this chapter

1. The safety gate: Filter, then Classifier
2. Token-cap delegation
3. Self-service token requests

6.1 The Safety Gate: Filter, then Classifier

Before any retrieved or uploaded text reaches a model (or a reply reaches a user), it passes through a two-stage gate composed in one place, `services/screening.py`, and called identically by the retrieval path and the upload path (Figure 6.1).

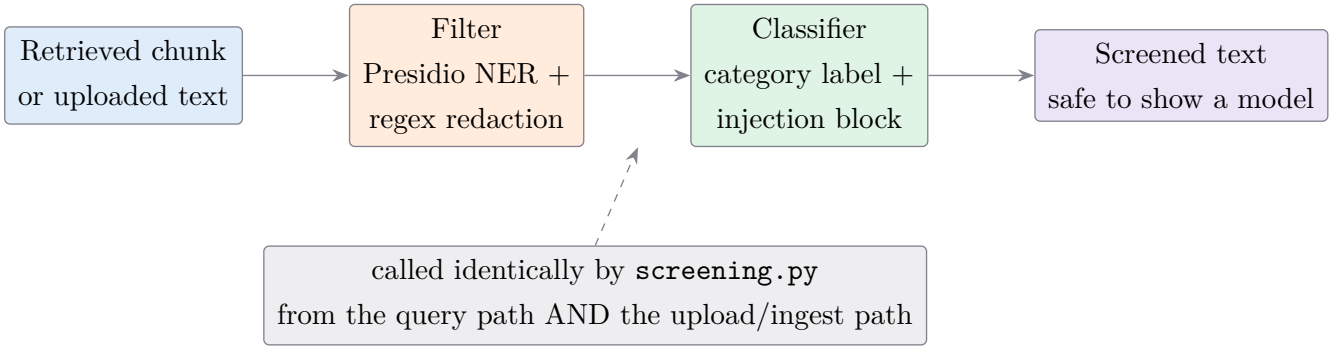


Figure 6.1: Every piece of text passes through the Filter, then the Classifier, before reaching a model.

1. **Filter** (`filter.py`): personal-data redaction built on **Microsoft Presidio** (built-in recognizers, a spaCy NER engine, and admin-editable custom patterns scoped globally or to one company), including checksum-validated national-ID recognizers for French, Spanish and Polish documents. Matches are replaced with `[REDACTED_<TYPE>]` markers; a regex fallback keeps redaction working even if Presidio itself errors.
2. **Classifier** (`classifier.py`): a lighter, rule-based stage that labels a chunk's category (billing, technical support, safety, HR, general) and drops prompt-injection attempts outright.

Person and location detection are implemented in the Filter but kept off the live entity list: the NER model gives every `PERSON` match a flat confidence score regardless of whether it is a real name, and the platform's knowledge base is full of capitalized product names that would score identically. Re-enabling it needs a better-calibrated model.

6.2 Token-Cap Delegation

Two independent caps are checked before every call: a whole-company daily budget and a per-user daily budget, with super administrators permanently uncapped since they have no one above them who could raise a cap they hit. A single authorization function states, once, who may change whose cap, and is called from every path that can change one, so the rule cannot be bypassed by a different route to the same field (Figure 6.2).

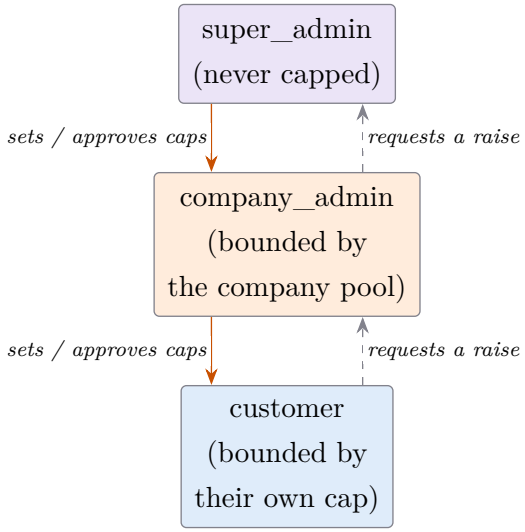


Figure 6.2: Caps flow down the ladder; raise requests flow back up it.

Every user cap is bounded by their own company's pool (a *per-cap* check, not a sum-of-caps one), and a stored value of `0` is deliberately treated as *unlimited* rather than *blocked*, so it is refused whenever the company itself has a finite pool, closing the one value that would otherwise escape the tenant ceiling.

6.3 Self-Service Token Requests

Delegation lets an authorized caller *change* a cap; it does not give a capped user a way to *ask*. `TokenRequest` adds that other direction: a customer requests a raise to their own cap from their company administrator (`scope='user'`), and a company administrator requests more of the whole tenant pool from a super administrator (`scope='company'`). The `scope` is always derived from the caller's own role, never taken from the request

body. A `user`-scope approval overwrites the target's cap outright, while a `company`-scope approval adds to whatever the pool already is, the same additive-versus-absolute distinction Figure 6.2 shows.

The additive rule is deliberate: a company administrator asking for "X more tokens a day" should not have to know the pool's current size, and re-filing a second request after an earlier grant must not silently undo it. The same validation function used by the direct `PATCH /api/users/<id>/token-limit` path is reused here, so the two paths cannot drift apart, and an undecided request of the same scope blocks a duplicate filing rather than silently stacking.

Chapter 7

Observability, Feedback and Extensibility

In this chapter

1. Answer-quality feedback dashboard
2. Tracing and prompt versioning with MLflow
3. Agentic admin tooling through the Model Context Protocol

7.1 Answer-Quality Feedback Dashboard

The chat interface lets a customer leave a thumbs up or down on any answer, written to `Message.feedback`. On its own that column only records opinion, not *which model* earned it; a nullable `Message.provider_id` foreign key closes that gap. From there, `services/feedback.py` aggregates votes into an overall satisfaction score, a per-provider and per-company breakdown, and a review queue of recent down-voted answers with their sources, letting an administrator tell a bad-retrieval failure from a bad-generation one. Figure 7.1 shows this path alongside the platform's other observation path, tracing. The score is a percentage of *rated* answers, left as `None` until at least one vote exists, so the UI can say "no votes" instead of a misleading "0%".

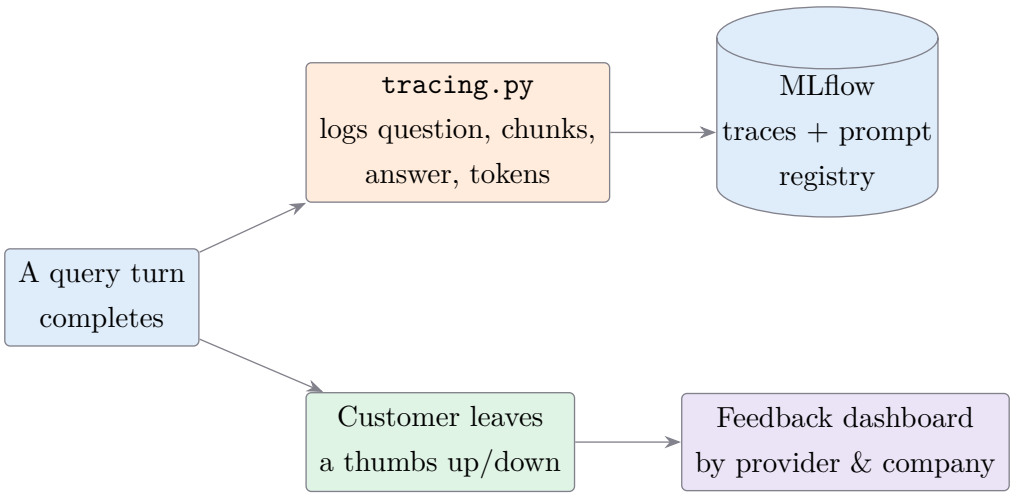


Figure 7.1: Two always-on paths, tracing and feedback, both keyed to the answering provider.

7.2 Tracing and Prompt Versioning with MLflow

Nothing else in the stack can answer, after the fact, whether a given answer was actually good, or what system prompt produced it. `services/ tracing.py` is the only module that imports the MLflow SDK, and it addresses both concerns: `record_trace()` logs one query's question, retrieved chunks, answer and token counts as a browsable MLflow trace, and `get_system_prompt()` loads a versioned, production-aliased prompt from MLflow's registry instead of a hardcoded string, falling back to that hardcoded constant whenever the registry has nothing registered yet. Every function in the module is wrapped in a broad exception handler: an unreachable or misconfigured MLflow server must never be able to break a real chat request.

```
def get_system_prompt(default: str) -> str:
    if not _enabled():
        return default
    try:
        import mlflow
        prompt =
            ↪ mlflow.genai.load_prompt("prompts:/rag-system-prompt@production")
        return prompt.template
    except Exception as e:
        current_app.logger.warning("mlflow get_system_prompt falling
            ↪ back: %s", e)
        return default
```

Listing 7.1: A registry lookup with a hardcoded fallback: MLflow being down degrades the prompt to its last known-good constant rather than failing the request

7.3 Agentic Admin Tooling via the Model Context Protocol

The last piece is less a feature than a new *surface*: letting an off-the-shelf AI agent (AnythingLLM) reach the platform's own operational data. `mcp-admin-server/` gives it the one job a hand-built console cannot do on its own, agentic multi-step tool use, by wrapping admin endpoints as MCP tools over the real HTTP API rather than importing backend code directly (Figure 7.2). Every tool is a thin wrapper around an existing admin endpoint: the agent gets exactly the same data and authorization an administrator would see in the console.

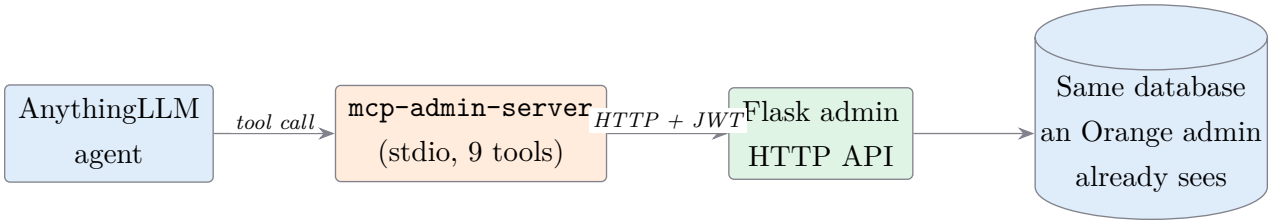


Figure 7.2: The MCP server is a thin wrapper: an agent can do only what an admin already could.

The server logs in like any admin user, re-authenticating on a 401, and talks to the Flask API over stdio-driven tool calls with no port of its own to manage. Its tools cover companies, providers, usage, feedback, PII rules, and the ingestion/deletion audit trail, plus one *active* tool that sends a real prompt to check whether a provider is actually responding, not merely marked enabled. A caller resolves a human-readable company or provider *name* to its internal id locally, so the agent's own dialogue stays in the vocabulary an administrator uses.

Conclusion

Put back together, the pieces in this report form one continuous path. A user logs in and is tied, server-side, to exactly one company (Chapter 3); their question is answered by a model chosen from a registry an administrator can change without a redeploy (Chapter 4); the answer is built only from that company's own documents, screened for personal data and prompt-injection on the way in and out, through a pipeline that degrades to "I don't have that information" rather than guess. None of these are bolted-together features: each exists because the layer below it enforces a guarantee the layer above relies on without re-checking, the route layer trusting the security layer's company resolution, the pipeline trusting the safety gate has already run, the provider layer trusting a cap has already cleared.

A few design decisions recur for the same underlying reason. Every sensitive concern (Qdrant access, outbound LLM calls, company resolution, key encryption) is confined to exactly one module, so a bug has exactly one place it could live. Where a feature could fail (a translation call, an unreachable MLflow server, a provider at capacity), the system degrades rather than breaks the request: untranslated retrieval, a hardcoded fallback prompt, or a clear error instead of a hang. And where a guarantee was already structurally true, tenant isolation being the clearest example, later hardening work stayed honest about not claiming to fix a leak that never existed, only adding a coarser layer on top.

Some corners remain open by design. Person and location redaction is implemented but switched off until a better-calibrated model can tell a real name from a capitalized product name. The per-cap token ceiling could extend to a hard sum-of-caps mode. A release path for a dedicated Qdrant slot would let a decommissioned tenant's instance be reassigned rather than held indefinitely. And a genuinely multilingual embedding model remains a valid, larger-scope alternative to query-time translation. Each would extend the same layered, single-chokepoint architecture, not replace it.

References

1. Flask: Web framework documentation. https://flask.palletsprojects.com/
2. Qdrant: Vector database documentation. https://qdrant.tech/documentation/
3. LiteLLM: Unified LLM SDK documentation. https://docs.litellm.ai/
4. Angular: Official documentation. https://angular.dev/
5. JSON Web Tokens: Introduction. https://jwt.io/introduction
6. Cryptography (Fernet): Documentation. https://cryptography.io/en/latest/fernet/
7. Retrieval-Augmented Generation: Overview. https://www.pinecone.io/learn/retrieval-augmented-generation/
8. MLflow: Documentation (tracking, tracing, prompt registry). https://mlflow.org/docs/latest/index.html
9. Microsoft Presidio: Data protection and de-identification documentation. https://microsoft.github.io/presidio/
10. Model Context Protocol: Specification. https://modelcontextprotocol.io/
11. AnythingLLM: Documentation. https://docs.anythingllm.com/